

Optimization Techniques: Exercise 1

Maria Charisi 10727

November 2025

Github Repo

Abstract

This report constitutes the first part of the assignment for the *Optimization Techniques* course at the *Aristotle University of Thessaloniki*. Its objective is to implement four different optimization methods in MATLAB to determine the minima of given convex functions. The report presents a brief theoretical description of each method, followed by the corresponding implementation details, graphical results, and a discussion of the observed performance of the algorithms.

Contents

1	Introduction	2
2	Dichotomy Method	2
2.1	Method Description	2
2.2	Plots and Observations	3
3	Golden Section Method	6
3.1	Method Description	6
3.2	Plots and Observations	7
4	Fibonacci Method	10
4.1	Method Description	10
4.2	Plots and Observations	11
5	Dichotomy Method with Derivatives	13
5.1	Method Description	13
5.2	Plots and Observations	14
6	Results	17
6.1	Comparison of the Four Methods	17
6.2	Convergence Speed	17
6.3	General Conclusions	18

1 Introduction

In this report, we study and compare four optimization methods applied to convex functions. Specifically, we consider three functions that are convex in the interval $[-1, 3]$:

$$f_1(x) = 5^x + (2 - \cos x)^2, \quad (1)$$

$$f_2(x) = (x - 1)^2 + e^{(x-5)} \sin(x + 3), \quad (2)$$

$$f_3(x) = e^{-3x} - (\sin(x - 2) - 2)^2. \quad (3)$$

We will apply and compare the following optimization techniques to estimate the minima of the given functions:

- Dichotomy Method
- Golden Section Method
- Fibonacci Method
- Dichotomy Method with Derivatives

A visual representation of the three functions within the interval $[-1, 3]$ is shown in Figure 1.

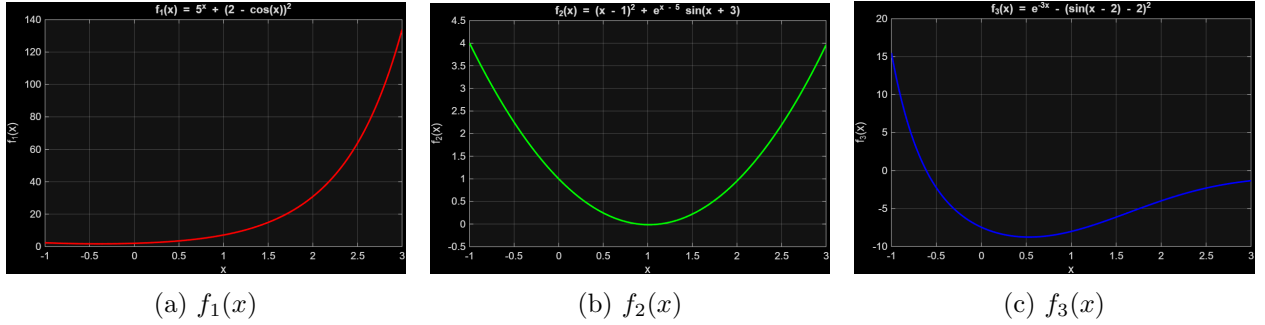


Figure 1: Plots of the three convex functions in the interval $[-1, 3]$.

2 Dichotomy Method

2.1 Method Description

The Dichotomy Method is a technique for locating the minimum of a function within a given interval $[a, b]$. The method iteratively reduces the search interval by evaluating the function at two points placed symmetrically around the midpoint.

At each iteration, two evaluation points are defined as:

$$x_1 = \frac{a + b}{2} - \varepsilon, \quad x_2 = \frac{a + b}{2} + \varepsilon,$$

where ε is a small positive constant ensuring that x_1 and x_2 are distinct. The function values $f(x_1)$ and $f(x_2)$ are compared:

If $f(x_1) < f(x_2)$, then $b = x_2$; otherwise, $a = x_1$.

This process repeats until the length of the current interval satisfies:

$$(b - a) < l,$$

where l is the desired precision. The midpoint of the final interval is taken as the estimated minimizer:

$$x_{\min} = \frac{a + b}{2}.$$

The following code, shown in Listing 1, presents the MATLAB implementation of the Dichotomy Method.

Listing 1: Implementation of the Dichotomy Method in MATLAB

```

1 function [min_x, min_f, num_calculations, k, ak_vector, bk_vector] =
   dichotomy_method(f, a, b, l, e)
2   num_calculations = 0;
3   k = 1;
4   ak_vector = a;
5   bk_vector = b;
6
7   while (b - a) > l
8       c = (a + b) / 2;
9       x1 = c - e;
10      x2 = c + e;
11
12      fx1 = f(x1);
13      fx2 = f(x2);
14      num_calculations = num_calculations + 2;
15
16      if fx1 < fx2
17          b = x2; % minimum in left half
18      elseif fx1 > fx2
19          a = x1; % minimum in right half
20      else
21          a = x1;
22          b = x2;
23      end
24
25      ak_vector = [ak_vector, a];
26      bk_vector = [bk_vector, b];
27      k = k + 1;
28   end
29
30   min_x = (a + b) / 2;
31   min_f = f(min_x);
32 end

```

2.2 Plots and Observations

In this subsection, we present the numerical results of applying the Dichotomy Method to the three test functions. The behavior of the method is analyzed by examining the number of function evaluations under different parameters, as well as the convergence of the interval endpoints.

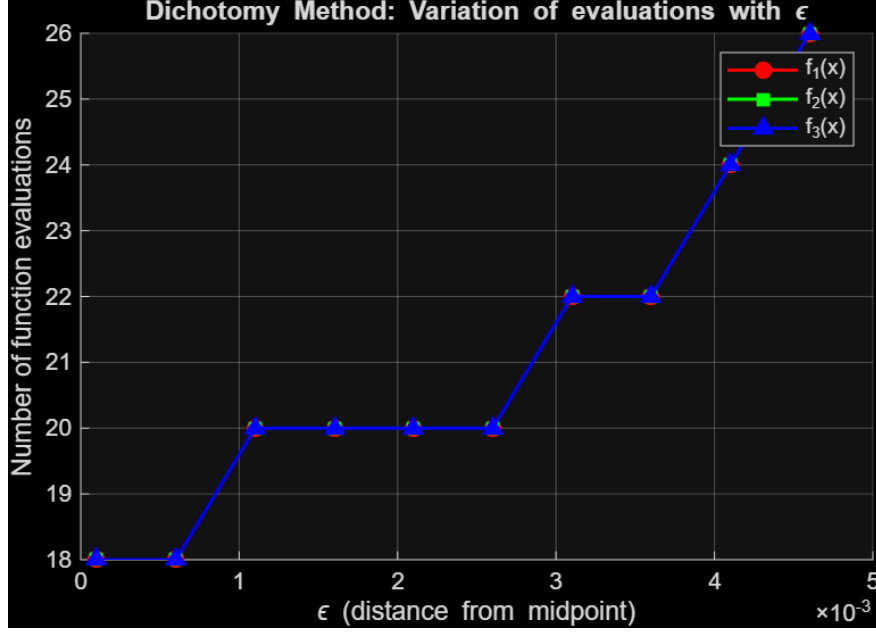


Figure 2: Number of function evaluations as a function of ε , with fixed $l = 0.01$.

Figure 2 shows the number of function evaluations required for each test function when the tolerance is fixed at $l = 0.01$ and ε varies from 0.0001 to 0.005. It can be observed that larger values of ε result in a higher number of function evaluations. This occurs because a larger ε increases the distance between the two evaluation points, which affects the interval reduction per iteration and leads to more iterations being needed to satisfy the stopping criterion.

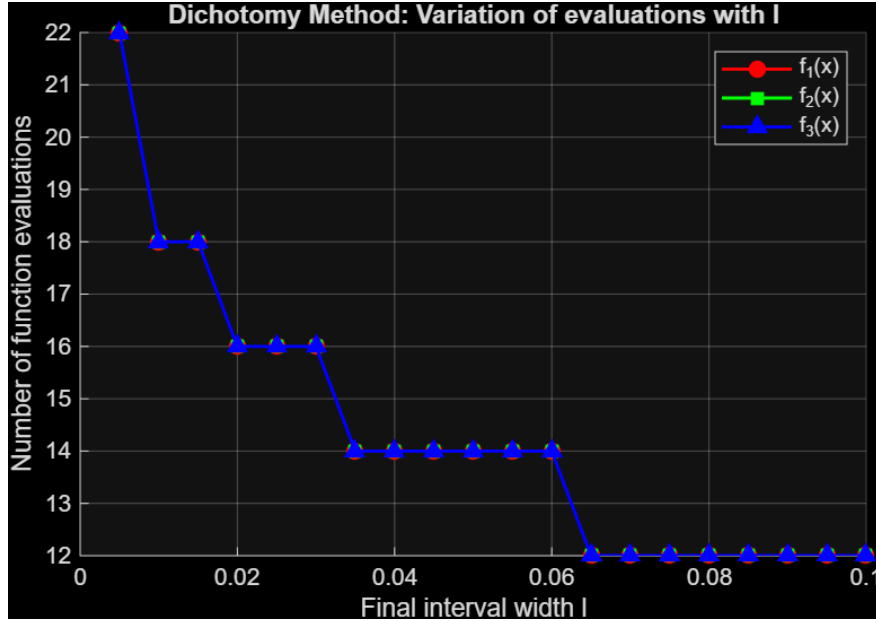


Figure 3: Number of function evaluations as a function of l , with fixed $\varepsilon = 0.001$.

Figure 3 illustrates how the number of function evaluations depends on the interval tolerance l , for a fixed $\varepsilon = 0.001$. Smaller tolerance values lead to more iterations and thus more function evaluations, which is consistent with the stopping criterion of the method.

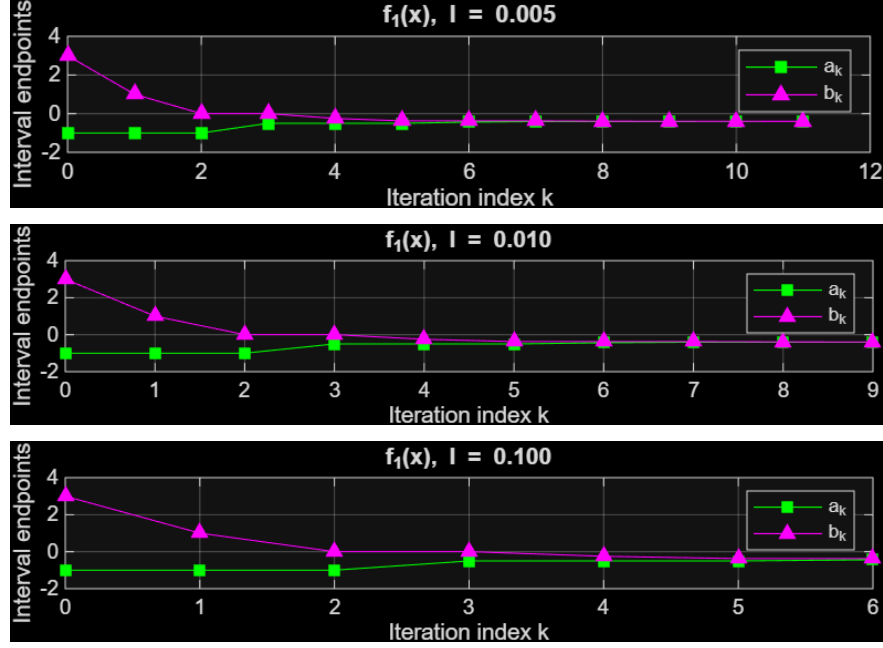


Figure 4: Convergence of interval endpoints a and b for $f_1(x)$ with Dichotomy Method, for different values of the tolerance l and $\varepsilon = 0.001$.

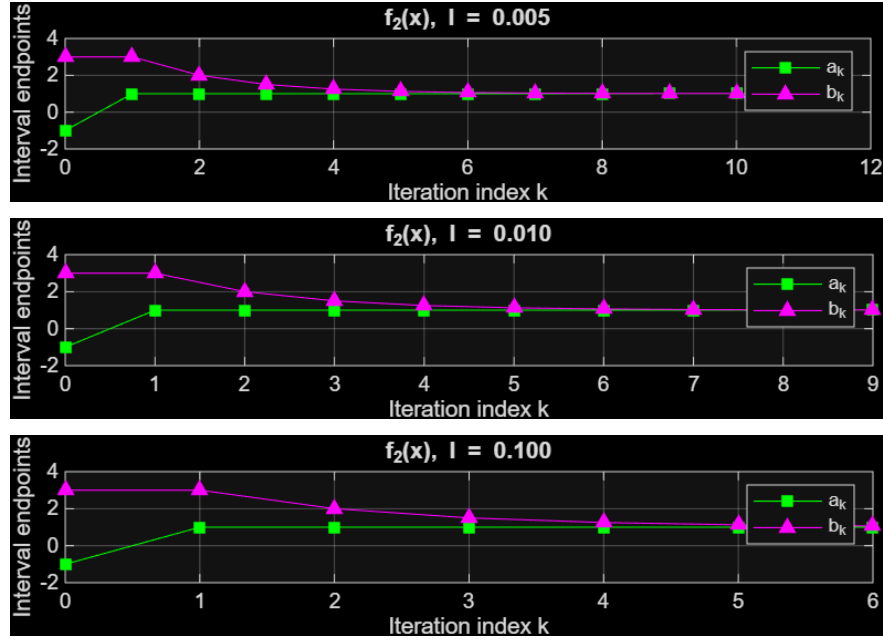


Figure 5: Convergence of interval endpoints a and b for $f_2(x)$ with Dichotomy Method, for different values of the tolerance l and $\varepsilon = 0.001$.

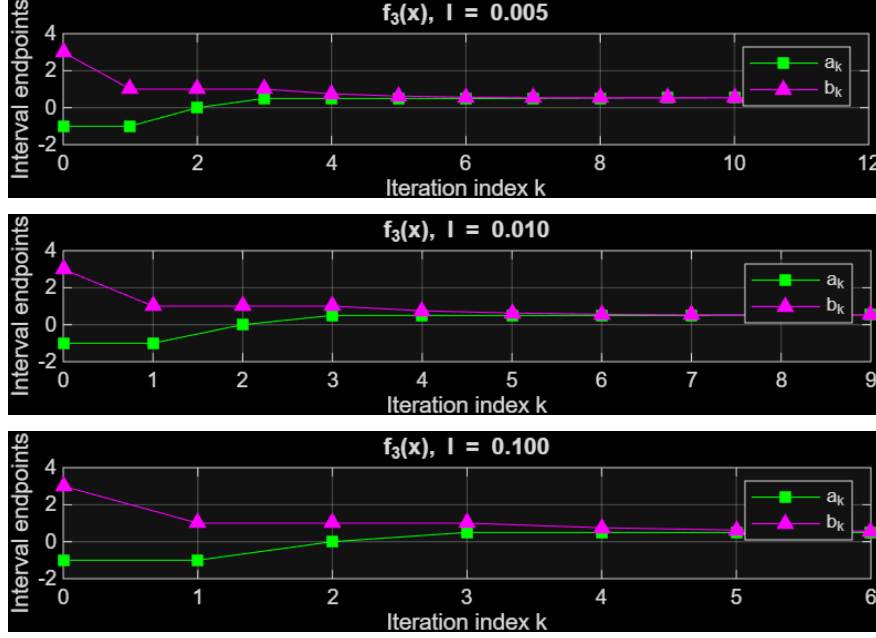


Figure 6: Convergence of interval endpoints a and b for $f_3(x)$ with Dichotomy Method, for different values of the tolerance l and $\varepsilon = 0.001$.

Figure 4–6 show the evolution of the interval endpoints a and b at each iteration for all three test functions, for different values of the tolerance l with $\varepsilon = 0.001$. It can be observed that for smaller values of l , the interval endpoints converge more slowly, requiring more iterations to reach the stopping criterion. Conversely, for larger values of l , the intervals converge faster, as the stopping condition is satisfied sooner.

3 Golden Section Method

3.1 Method Description

The Golden Section Method is an optimization technique for finding the minimum of a function within a given interval $[a, b]$. It reduces the search interval iteratively using the golden ratio $\phi = \frac{\sqrt{5}-1}{2} \approx 0.618$, which allows reusing function evaluations from previous iterations efficiently.

At each iteration, two points are chosen inside the current interval:

$$x_1 = b - \phi(b - a), \quad x_2 = a + \phi(b - a),$$

and the function values $f(x_1)$ and $f(x_2)$ are computed. The interval is updated depending on the comparison:

$$\text{If } f(x_1) < f(x_2), \text{ then } b = x_2, \quad \text{otherwise } a = x_1.$$

This process continues until the interval satisfies the stopping condition:

$$(b - a) < l,$$

where l is the desired tolerance. The midpoint of the final interval is taken as the estimated minimizer:

$$x_{\min} = \frac{a + b}{2}.$$

The following code, shown in Listing 2, presents the MATLAB implementation of the Golden Section Method.

Listing 2: Implementation of the Golden Section Method in MATLAB

```

1 function [min_x, min_f, num_calculations, k, ak_vector, bk_vector] =
   golden_section(f, a, b, l)
2
3     phi = (sqrt(5)-1)/2;    % Golden ratio (~0.618)
4     k = 1;
5     ak_vector = a;
6     bk_vector = b;
7     num_calculations = 0;
8
9     x1 = b - phi*(b-a);
10    x2 = a + phi*(b-a);
11    f1 = f(x1);
12    f2 = f(x2);
13    num_calculations = num_calculations + 2;
14
15    while (b - a) > l
16        if f1 < f2
17            b = x2;
18            x2 = x1;
19            f2 = f1;
20            x1 = b - phi*(b-a);
21            f1 = f(x1);
22        else
23            a = x1;
24            x1 = x2;
25            f1 = f2;
26            x2 = a + phi*(b-a);
27            f2 = f(x2);
28        end
29
30        ak_vector = [ak_vector, a];
31        bk_vector = [bk_vector, b];
32        num_calculations = num_calculations + 1;
33        k = k + 1;
34    end
35
36    min_x = (a+b)/2;
37    min_f = f(min_x);
38 end

```

3.2 Plots and Observations

The numerical results of the Golden Section Method applied to the three test functions are presented below. We focus on the number of function evaluations as a function of the interval tolerance l , along with the convergence behavior of the interval endpoints.

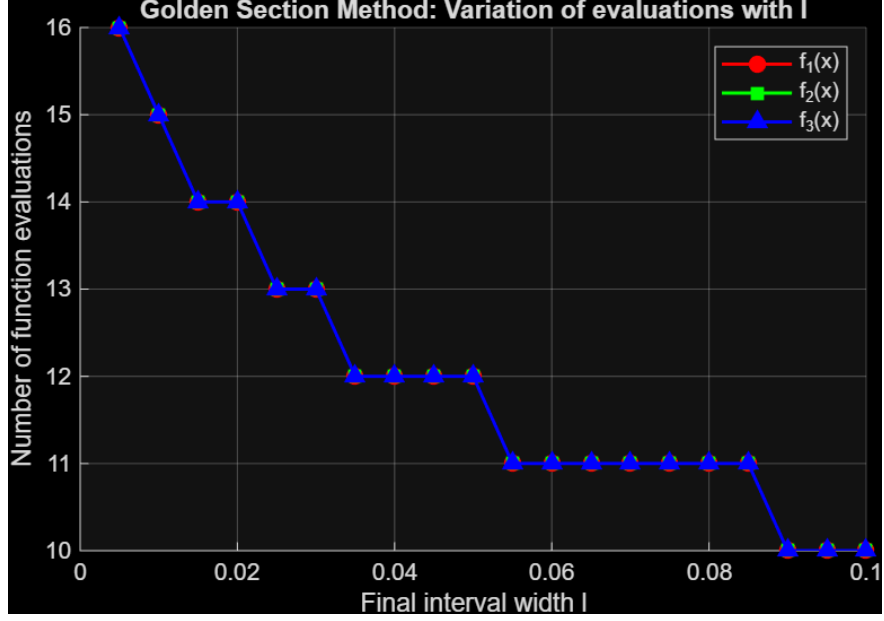


Figure 7: Number of function evaluations as a function of l , with the Golden Section Method.

Figure 7 shows that smaller values of l require more function evaluations, as the stopping criterion demands more iterations for tighter tolerances.

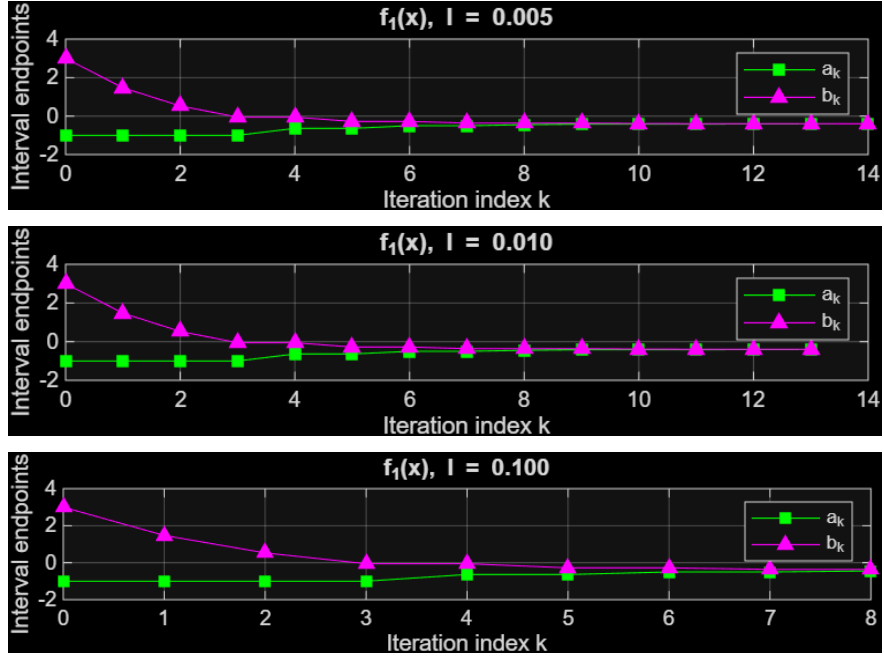


Figure 8: Convergence of interval endpoints a and b for $f_1(x)$ with the Golden Section Method, for different values of the tolerance l .

Figure 8–10 show the evolution of the interval endpoints a and b for all three functions. As with the Dichotomy Method, smaller tolerance l leads to slower convergence of the interval endpoints, requiring more iterations, while larger l values result in faster convergence.

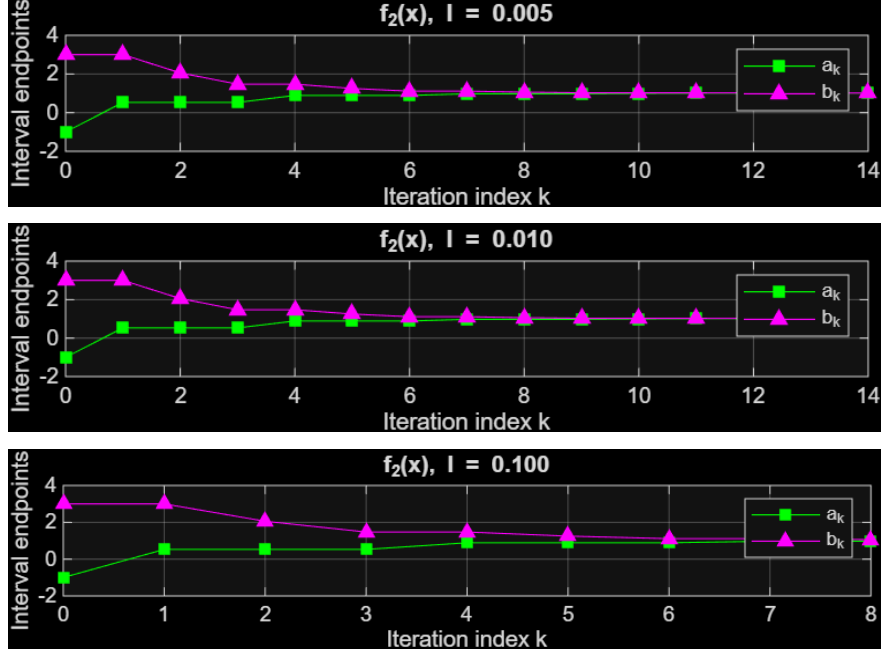


Figure 9: Convergence of interval endpoints a and b for $f_2(x)$ with the Golden Section Method, for different values of the tolerance l .

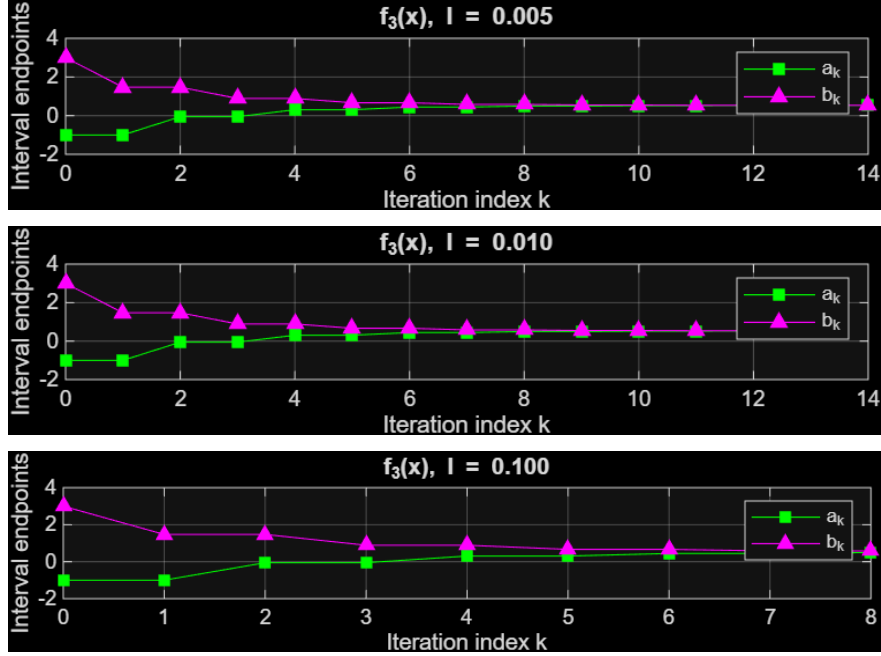


Figure 10: Convergence of interval endpoints a and b for $f_3(x)$ with the Golden Section Method, for different values of the tolerance l .

4 Fibonacci Method

4.1 Method Description

The Fibonacci Method is an iterative optimization technique used to locate the minimum of a function within a given interval $[a, b]$. It is similar in principle to the Golden Section Method, but it chooses evaluation points based on ratios of consecutive Fibonacci numbers to minimize the number of function evaluations.

The method generates a sequence of Fibonacci numbers $F_1, F_2, \dots, F_n$ such that

$$F_n > \frac{b-a}{l},$$

where l is the desired interval tolerance. At each iteration, two interior points are selected as:

$$x_1 = a + \frac{F_{n-2}}{F_n}(b-a), \quad x_2 = a + \frac{F_{n-1}}{F_n}(b-a),$$

and the function values $f(x_1)$ and $f(x_2)$ are compared. Depending on the comparison, the interval is updated by discarding the subinterval that does not contain the minimum, and new points are calculated using the ratios of the remaining Fibonacci numbers.

The process continues until the interval length satisfies the stopping criterion:

$$(b-a) < l.$$

The midpoint of the final interval is taken as the estimated minimizer:

$$x_{\min} = \frac{a+b}{2}.$$

The following code, shown in Listing 3, presents the MATLAB implementation of the Fibonacci Method.

Listing 3: Implementation of the Fibonacci Method in MATLAB

```
1 function [min_x, min_f, num_calculations, k, ak_vector, bk_vector] =  
   fibonacci_method(f, a, b, l)  
2  
3   % Generate Fibonacci numbers until F(n) > (b-a)/l  
4   F = [1, 1];  
5   while F(end) < (b-a)/l  
6       F = [F, F(end) + F(end-1)];  
7   end  
8   n = length(F);  
9  
10  ak_vector = a;  
11  bk_vector = b;  
12  num_calculations = 0;  
13  
14  % Initial interior points  
15  x1 = a + F(n-2)/F(n)*(b-a);  
16  x2 = a + F(n-1)/F(n)*(b-a);  
17  f1 = f(x1);  
18  f2 = f(x2);  
19  num_calculations = num_calculations + 2;  
20  
21  for iter= 2:n-2
```

```

22     if f1 < f2
23         b = x2;
24         x2 = x1;
25         f2 = f1;
26         x1 = a + F(n-iter-1)/F(n-iter+1)*(b-a);
27         f1 = f(x1);
28     else
29         a = x1;
30         x1 = x2;
31         f1 = f2;
32         x2 = a + F(n-iter)/F(n-iter+1)*(b-a);
33         f2 = f(x2);
34     end
35
36     ak_vector = [ak_vector, a];
37     bk_vector = [bk_vector, b];
38     num_calculations = num_calculations + 1;
39 end
40
41 k = length(ak_vector);
42 min_x = (a+b)/2;
43 min_f = f(min_x);
44 end

```

4.2 Plots and Observations

The numerical results of the Fibonacci Method for the three test functions are presented below. The analysis focuses on the number of function evaluations as a function of the interval tolerance l , along with the convergence of the interval endpoints.

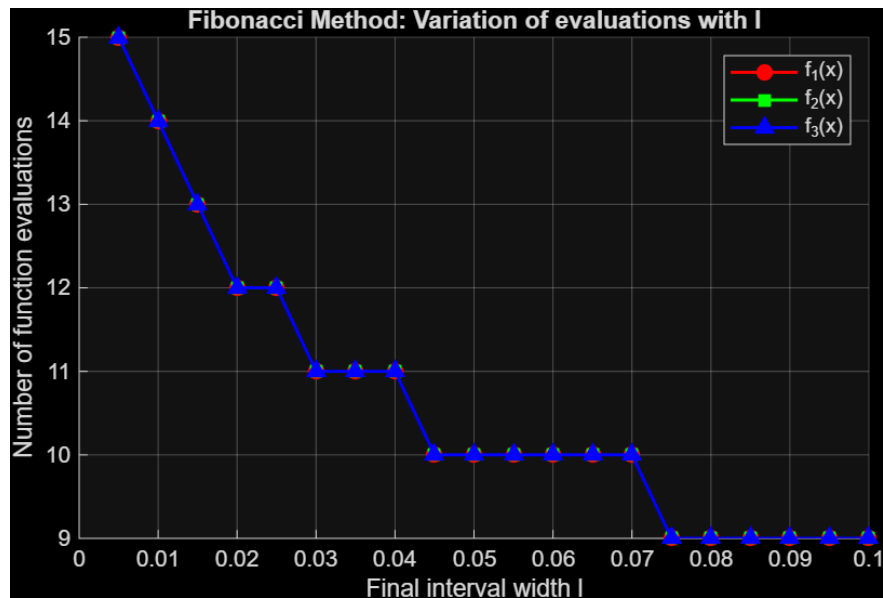


Figure 11: Number of function evaluations as a function of l , with the Fibonacci Method.

Figure 11 shows that smaller values of l require more function evaluations, as the stopping criterion demands more iterations for tighter tolerances.

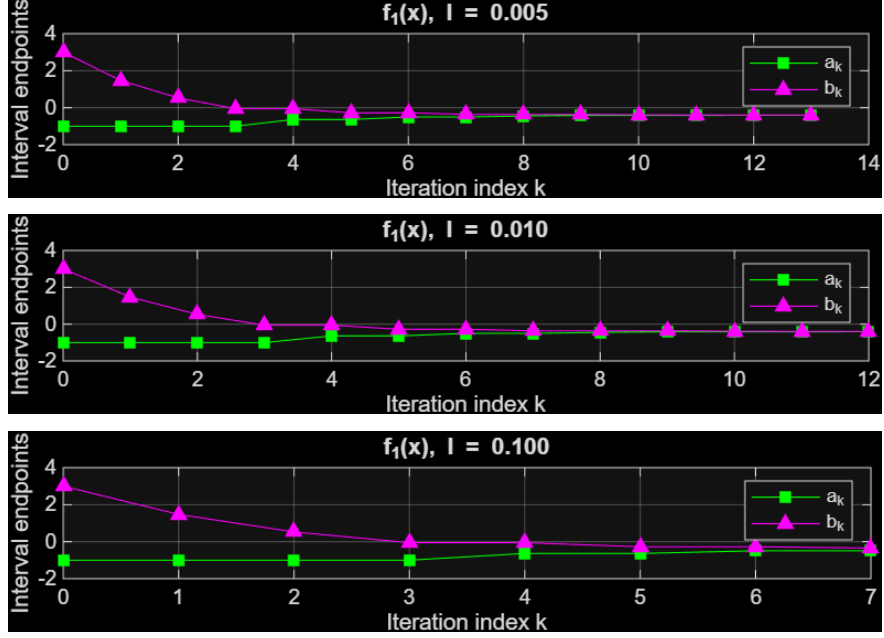


Figure 12: Convergence of interval endpoints a and b for $f_1(x)$ with the Fibonacci Method, for different values of the tolerance l .

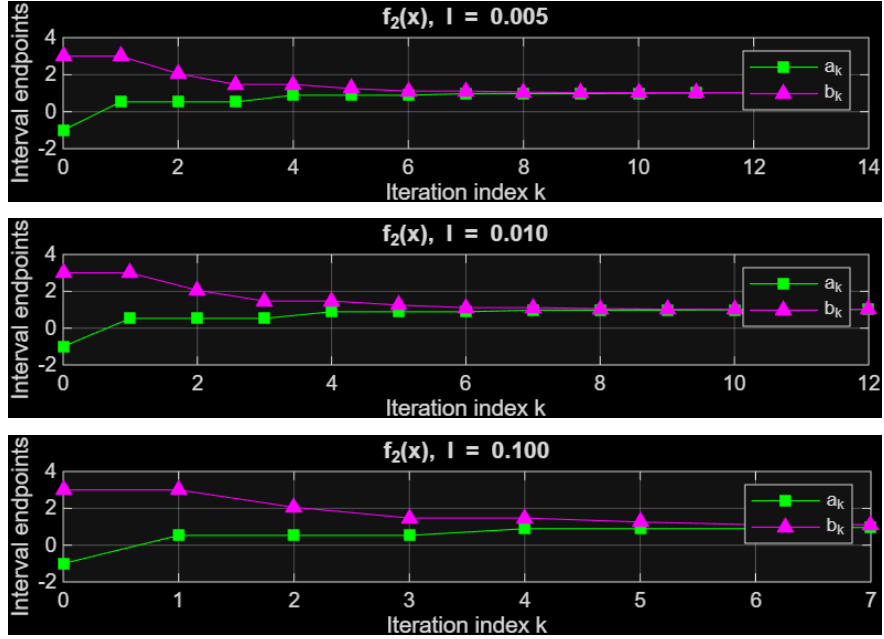


Figure 13: Convergence of interval endpoints a and b for $f_2(x)$ with the Fibonacci Method, for different values of the tolerance l .

Figure 12–14 show the evolution of the interval endpoints a and b for all three functions. As with the Dichotomy and Golden Section Methods, smaller tolerance l leads to slower convergence of the interval endpoints, requiring more iterations, while larger l values result in faster convergence.

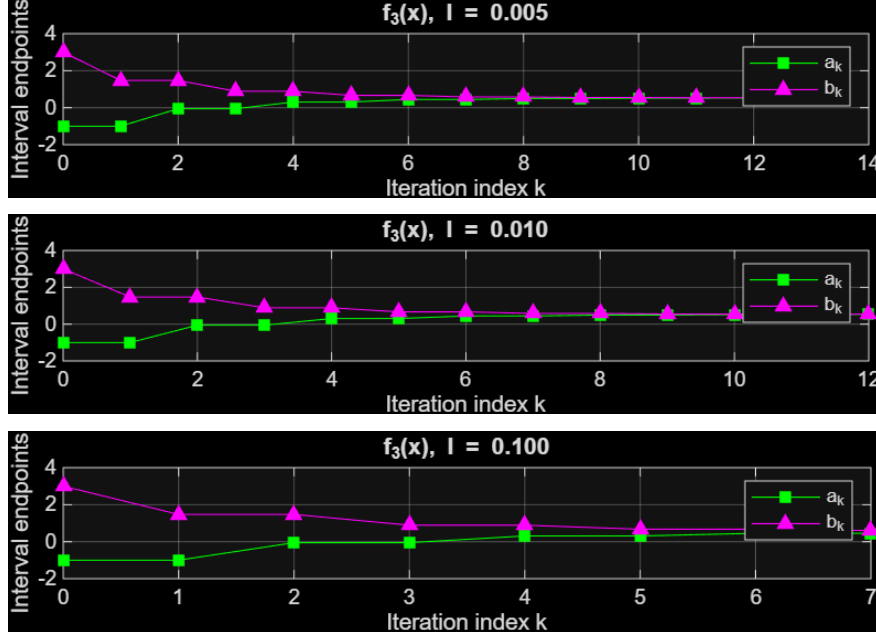


Figure 14: Convergence of interval endpoints a and b for $f_3(x)$ with the Fibonacci Method, for different values of the tolerance l .

5 Dichotomy Method with Derivatives

5.1 Method Description

The Dichotomy Method with Derivatives is a variant of the standard Dichotomy Method that uses the first derivative of the function to accelerate convergence. Instead of evaluating the function at two points around the midpoint, the method checks the sign of the derivative at the midpoint to determine the direction in which the minimum lies.

At each iteration, the midpoint of the current interval is computed:

$$c = \frac{a + b}{2}.$$

The derivative at the midpoint, $f'(c)$, is then evaluated. The interval is updated according to the sign of the derivative:

If $f'(c) < 0$, then the minimum lies to the right, set $a = c$,

If $f'(c) > 0$, then the minimum lies to the left, set $b = c$.

The iteration continues until the interval length satisfies the tolerance:

$$(b - a) < l.$$

The midpoint of the final interval is taken as the estimated minimizer:

$$x_{\min} = \frac{a + b}{2}.$$

The following code, shown in Listing 4, presents a MATLAB implementation of the Dichotomy Method with Derivatives.

Listing 4: Implementation of the Dichotomy Method with Derivatives in MATLAB

```

1 function [min_x, min_f, num_calculations, k, ak_vector, bk_vector] =
   dichotomy_derivative(f, df, a, b, l)
2     ak_vector = a;
3     bk_vector = b;
4     num_calculations = 0;
5     k = 1;
6
7     while (b - a) > l
8         c = (a+b)/2;
9         dfc = df(c);
10        num_calculations = num_calculations + 1;
11
12        if dfc < 0
13            a = c; % minimum in right half
14        elseif dfc > 0
15            b = c; % minimum in left half
16        else
17            a = c;
18            b = c;
19        end
20
21        ak_vector = [ak_vector, a];
22        bk_vector = [bk_vector, b];
23        k = k + 1;
24    end
25
26    min_x = (a+b)/2;
27    min_f = f(min_x);
28 end

```

5.2 Plots and Observations

The numerical results of the Dichotomy Method with Derivatives for the three test functions are presented below. The analysis focuses on the number of function evaluations as a function of the interval tolerance l , along with the convergence of the interval endpoints.

Figure 15 shows that smaller values of l require more function evaluations, as the stopping criterion demands more iterations for tighter tolerances.

Figure 16–18 show the evolution of the interval endpoints a and b for all three functions. As in the standard Dichotomy Method, smaller tolerance l leads to slower convergence of the interval endpoints, requiring more iterations, while larger l values result in faster convergence. Using the derivative accelerates convergence compared to the standard Dichotomy Method, as fewer function evaluations are needed to determine the interval update direction.

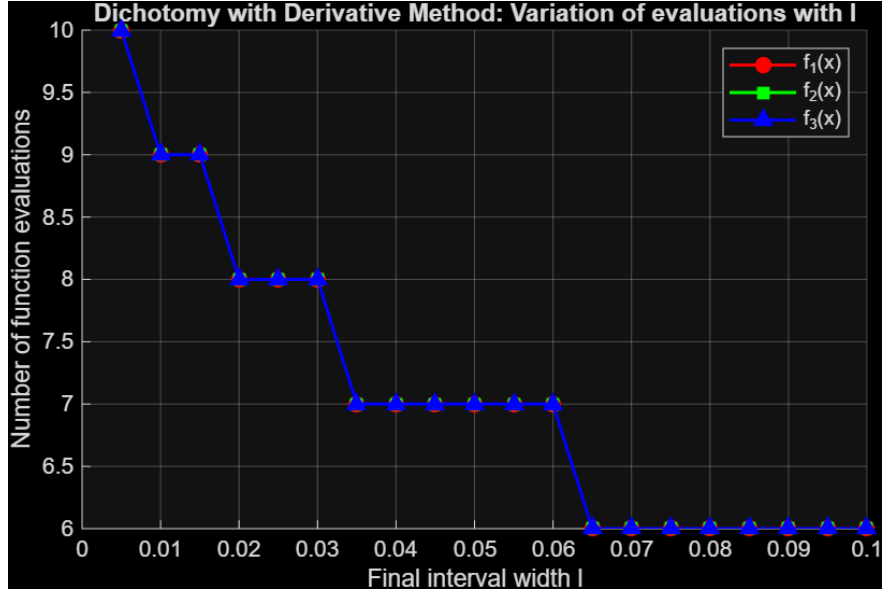


Figure 15: Number of function evaluations as a function of l , using the Dichotomy Method with Derivatives.

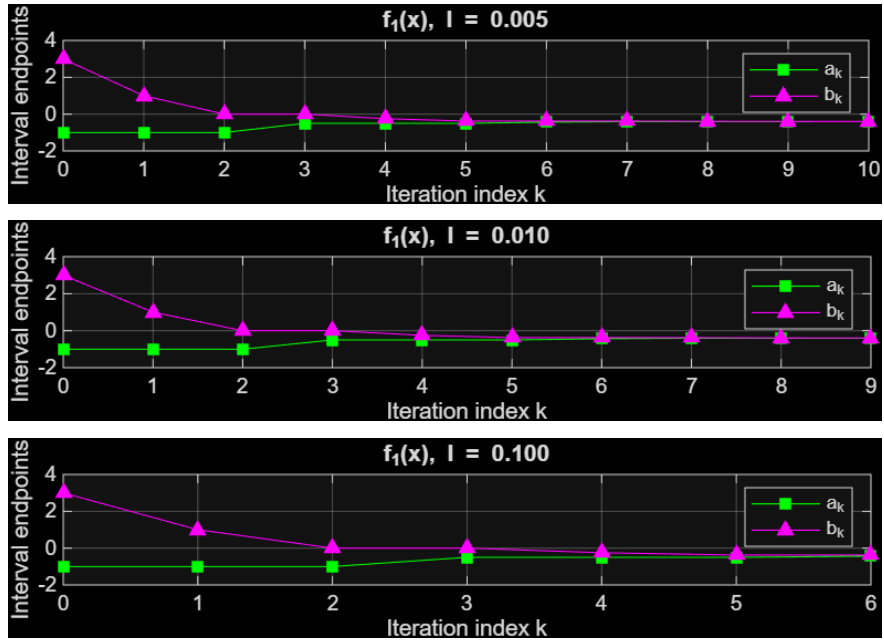


Figure 16: Convergence of interval endpoints a and b for $f_1(x)$ with the Dichotomy Method using Derivatives, for different values of the tolerance l .

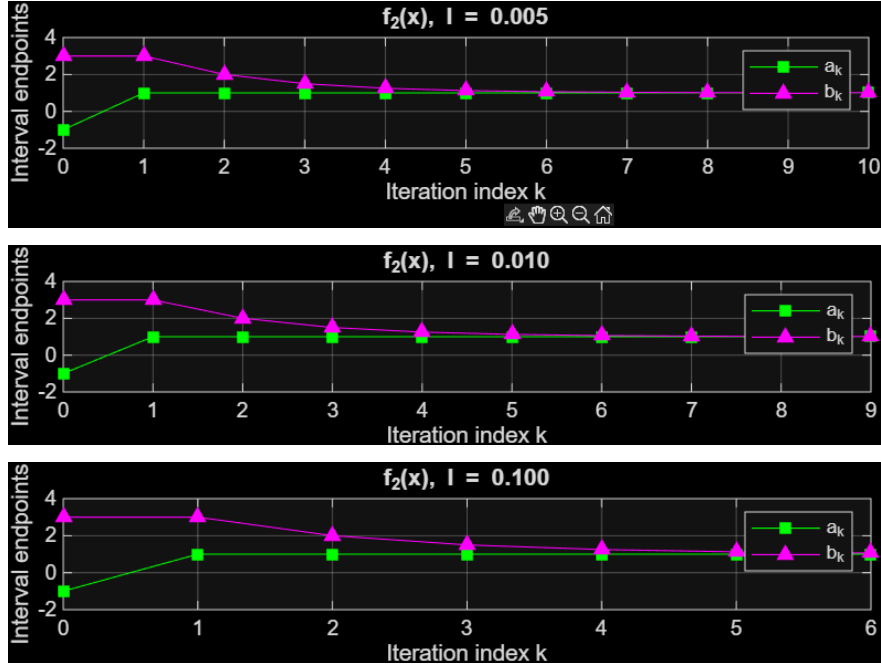


Figure 17: Convergence of interval endpoints a and b for $f_2(x)$ with the Dichotomy Method using Derivatives, for different values of the tolerance l .

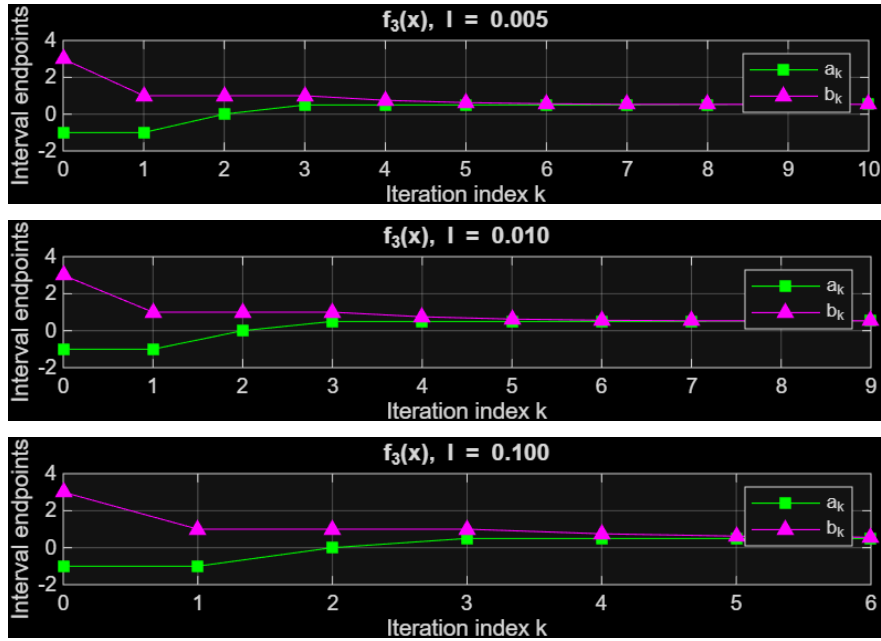


Figure 18: Convergence of interval endpoints a and b for $f_3(x)$ with the Dichotomy Method using Derivatives, for different values of the tolerance l .

6 Results

Table 1 and Table 2 summarize the number of iterations and the number of function evaluations, respectively, required by each optimization method to converge for three different values of the interval tolerance l .

Method	$l = 0.005$	$l = 0.01$	$l = 0.1$
Dichotomy Method	11	9	6
Golden Section Method	14	13	8
Fibonacci Method	13	12	7
Dichotomy Method with Derivatives	10	9	6

Table 1: Number of iterations required for convergence for each method and tolerance value l .

Method	$l = 0.005$	$l = 0.01$	$l = 0.1$
Dichotomy Method	22	18	12
Golden Section Method	15	14	9
Fibonacci Method	14	13	8
Dichotomy Method with Derivatives	10	9	6

Table 2: Number of function evaluations required for convergence for each method and tolerance value l .

6.1 Comparison of the Four Methods

The results presented in Tables 2 allow us to compare the behavior of the four optimization methods with respect to the number of iterations and the number of function evaluations required for convergence.

First, regarding the computational cost per iteration:

- **Dichotomy Method** evaluates the function twice per iteration (at the symmetrically placed points x_1 and x_2).
- **Golden Section Method** and **Fibonacci Method** evaluate the function twice only in the first iteration, and then only once per iteration, since they reuse a previously computed function value in each update step.
- **Dichotomy Method with Derivatives** evaluates the derivative only once per iteration, making each iteration cheaper compared to the standard Dichotomy Method.

These differences in per-iteration cost explain the results observed in the second table, where the total number of function evaluations differs significantly even when the number of iterations is similar.

6.2 Convergence Speed

A consistent trend across all values of the tolerance l is observed:

- **The Dichotomy Method with Derivatives converges faster** than the other three methods, both in terms of iterations and function evaluations. This is expected, since using derivative information allows the method to determine the search direction more efficiently.
- **Golden Section and Fibonacci Methods** exhibit very similar convergence behavior, both in iterations and function evaluations. This similarity arises because both methods use a fixed reduction ratio for updating the interval:
 - the Golden Section Method uses the golden ratio factor,
 - the Fibonacci Method uses ratios of consecutive Fibonacci numbers, which converge to the golden ratio as $n \rightarrow \infty$.

Therefore, both methods reduce the interval in a comparable way.

6.3 General Conclusions

- For stricter tolerances (smaller l), all methods require more iterations and function evaluations, as expected.
- Among the **derivative-free** methods, the Golden Section and Fibonacci Methods are more efficient than the standard Dichotomy Method.
- The **Dichotomy Method with Derivatives** is the most efficient overall, consistently achieving convergence with the smallest number of function evaluations.